

19. 删除链表的倒数第 N 个结点

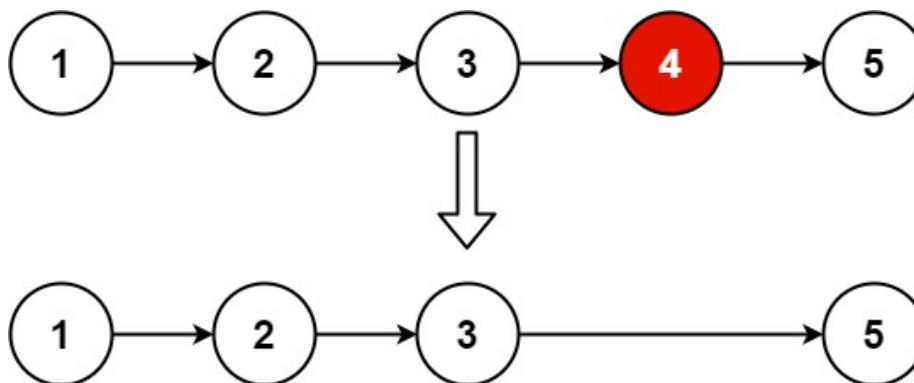
删除链表的倒数第 N 个结点

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium (50.77%)	3100	0	-	-	0

- Tags
- Companies

给你一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

示例 1:



输入: `head = [1,2,3,4,5]`, `n = 2`

输出: `[1,2,3,5]`

示例 2:

输入: `head = [1]`, `n = 1`

输出: `[]`

示例 3:

输入: `head = [1,2]`, `n = 1`

输出: `[1]`

提示:

- 链表中结点的数目为 `sz`
- $1 \leq sz \leq 30$
- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq sz$

进阶: 你能尝试使用一趟扫描实现吗?

2. 方法

LeetCode 19: 删除链表的倒数第 N 个结点（详细解析）

1. 问题理解

目标：从一个单链表中删除倒数第 n 个结点。

输入：

- 单链表的头节点 `head`
- 一个整数 n (表示要删除的是倒数第几个结点)

输出：删除指定结点后，链表的头节点。

示例：

- 输入： `head = [1,2,3,4,5]`, $n = 2$
- 输出： `[1,2,3,5]` (删除了倒数第 2 个结点，即 4)

关键点：

- “倒数第 n 个”是核心。
- 需要处理删除头结点的边界情况。
- 进阶要求：尝试只用一趟扫描完成。

2. 普通人的思路（及遇到的困难）

思路一：两次遍历

想法：

1. 先遍历一次链表，计算出链表的总长度 L 。
2. 倒数第 n 个结点，就是正数第 $L - n + 1$ 个结点。
3. 再遍历一次链表，找到第 $L - n$ 个结点（即要删除节点的前一个节点），然后执行删除操作。

实现步骤：

1. 遍历链表计算长度 L 。
2. 计算要找的目标节点的前一个节点的位置 `target_prev_index = L - n`。
3. 从头开始遍历，找到第 `target_prev_index` 个节点 `prev_node`。
4. 执行删除： `prev_node.next = prev_node.next.next`。
5. 特殊情况：如果要删除的是头节点（即 $n == L$ ），需要特殊处理，直接返回 `head.next`。

复杂度：

- 时间复杂度： $O(L)$ ，需要遍历链表两次。

- 空间复杂度: $O(1)$ 。

缺点: 需要遍历两次链表, 不满足进阶要求的一趟扫描。

3. 更好的方法 (一趟扫描: 快慢指针)

核心思想: 使用两个指针, **fast** 和 **slow**。让 **fast** 指针先走 n 步, 然后 **fast** 和 **slow** 一起走。当 **fast** 指针到达链表末尾时, **slow** 指针正好指向倒数第 $n+1$ 个节点 (即要删除节点的前一个节点)。

为什么需要哑节点 (**Dummy Node**)?

- 考虑删除头节点的情况 (例如 $[1]$, $n=1$ 或 $[1,2]$, $n=2$)。如果 **slow** 从 **head** 开始, 当 **fast** 到达末尾时, **slow** 会指向要删除的头节点, 我们无法方便地获取头节点的前一个节点来执行删除。
- 引入一个哑节点 **dummy**, 让它指向 **head**, 并让 **slow** 和 **fast** 都从 **dummy** 开始。这样, 即使要删除的是头节点, **slow** 最终也会停在 **dummy** 节点, 可以通过 `dummy.next = dummy.next.next` 来正确删除头节点。

步骤

1. 创建一个哑节点 **dummy**, 使其 **next** 指向 **head**。
2. 初始化快慢指针 **fast** 和 **slow** 都指向 **dummy**。
3. 让 **fast** 指针先向前移动 n 步。
4. 同时移动 **fast** 和 **slow** 指针, 每次都移动一步, 直到 **fast** 到达链表的最后一个节点 (即 `fast.next == null`)。
5. 此时, **slow** 指针指向的是倒数第 $n+1$ 个节点 (要删除节点的前一个节点)。
6. 执行删除操作: `slow.next = slow.next.next`。
7. 返回 `dummy.next` 作为结果链表的头节点。

4. 示例演示

以 `head = [1,2,3,4,5]`, $n = 2$ 为例:

1. 创建哑节点: `dummy -> 1 -> 2 -> 3 -> 4 -> 5`
2. 初始化指针: `fast = dummy, slow = dummy`
3. 快指针先行 $n=2$ 步:
 - **fast** 移动 1 步到节点 1
 - **fast** 移动 1 步到节点 2
 - 此时: **slow** 在 **dummy**, **fast** 在 2
4. 同时移动:
 - **fast** 到 3, **slow** 到 1
 - **fast** 到 4, **slow** 到 2
 - **fast** 到 5, **slow** 到 3
 - `fast.next == null`, 停止移动。
5. 定位: **slow** 指向节点 3, 这是倒数第 $n+1 = 3$ 个节点, 也是要删除的节点 4 的前一个节点。
6. 删除:

- `slow.next` (即节点 4)
 - `slow.next.next` (即节点 5)
 - 执行 `slow.next = slow.next.next`, 即 `3.next = 5`。
 - 链表变为: `dummy -> 1 -> 2 -> 3 -> 5`
7. 返回: `dummy.next`, 即节点 1。最终结果链表为 `[1,2,3,5]`。

5. 代码实现

```
java
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    /**
     * 删除链表的倒数第 N 个结点 (一趟扫描实现)
     * 方法: 快慢指针 + 哑节点
     * @param head 链表头节点
     * @param n 要删除的倒数第 n 个节点
     * @return 删除节点后的链表头节点
     */
    public ListNode removeNthFromEnd(ListNode head, int n) {
        // 1. 创建哑节点, 处理删除头节点的边界情况
        ListNode dummy = new ListNode(0);
        dummy.next = head;

        // 2. 初始化快慢指针都指向哑节点
        ListNode fast = dummy;
        ListNode slow = dummy;

        // 3. 快指针先向前移动 n 步
        // 使得快慢指针之间相隔 n 个节点
        for (int i = 0; i < n; i++) {
            // 题目保证 1 <= n <= sz, 所以 fast 不会在此过程中变为 null
            fast = fast.next;
        }

        // 4. 同时移动快慢指针, 直到快指针到达链表末尾
        // 当 fast.next == null 时, fast 指向最后一个节点
        // 此时 slow 指向的是倒数第 n+1 个节点 (即要删除节点的前一个节点)
```

```

        while (fast.next != null) {
            fast = fast.next;
            slow = slow.next;
        }

        // 5. 删除倒数第 n 个节点
        // slow 指向要删除节点的前一个节点
        // slow.next 指向要删除的节点
        // slow.next.next 指向要删除节点的后一个节点
        slow.next = slow.next.next;

        // 6. 返回哑节点的下一个节点，即为处理后的链表头节点
        return dummy.next;
    }
}

```

6. 代码解析

- 哑节点 **dummy**: 简化了删除头节点的操作，使得所有删除操作逻辑统一。
- 快指针 **fast** 先行 **n** 步: 这是关键，确保了 **fast** 和 **slow** 之间始终保持 **n** 个节点的距离。
- 同时移动: 当 **fast** 到达链表末尾时，利用 **n** 的距离差，**slow** 自然就定位到了目标节点（要删除节点）的前一个节点。
- 删除操作 **slow.next = slow.next.next**: 标准的链表节点删除方式，跳过 **slow.next** 指向的节点。
- 返回 **dummy.next**: 保证了即使原始头节点被删除，也能返回正确的新的头节点。

7. 复杂度分析

- 时间复杂度: $O(L)$ ，其中 L 是链表的长度。虽然有两个指针，但它们只对链表进行了一次完整的遍历。
- 空间复杂度: $O(1)$ ，只使用了常数个额外的指针变量 (**dummy**, **fast**, **slow**)。

8. 如何在面试中讲解

1. 确认问题和要求:
 - “目标是删除链表的倒数第 n 个节点，并返回头节点。进阶要求是一趟扫描完成。”
2. 讨论简单但不够优化的方法:
 - “可以先遍历一次计算长度 L ，再遍历一次找到第 $L-n$ 个节点进行删除。但这需要两次遍历。”
3. 引入一趟扫描的解法（快慢指针）:
 - “为了实现一趟扫描，我们可以使用快慢指针。核心思想是让快指针先走 n 步，然后两个指针一起走。”
4. 解释哑节点的作用:

- “为了方便处理删除头节点的边界情况，我们引入一个哑节点 `dummy`，让它指向 `head`，所有指针都从 `dummy` 开始。”

5. 详细描述算法流程：

- “初始化 `fast` 和 `slow` 指向 `dummy`。”
- “让 `fast` 先走 `n` 步。”
- “然后同时移动 `fast` 和 `slow`，直到 `fast` 到达链表最后一个节点 (`fast.next == null`)。”
- “此时 `slow` 指向的就是要删除节点的前一个节点。”
- “执行删除操作 `slow.next = slow.next.next`。”
- “最后返回 `dummy.next`。”

6. 强调效率和正确性：

- “这个方法只需要一趟扫描，时间复杂度是 $O(L)$ 。”
- “空间复杂度是 $O(1)$ 。”
- “哑节点确保了即使删除头节点也能正确处理。”

找到具有 1 个许可证类型的类似代码